

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 3 - Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)
Weightage:	10% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	Ch Rama Karthik Reddy	Reg. No.:	192472281
Section / Batch:	2024	Date:	

Instructions to Students

- Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.
- Develop and execute the assigned **Python program** using Text-to-Image, Speech-to-Text, or Text-to-Speech.
- Explain the **working principle and program workflow**.
- Demonstrate the program with **suitable inputs and outputs**.
- Use appropriate **libraries/APIs** and include basic **error handling**.
- Show the **execution output/screenshots**.
- For integrated tasks, demonstrate the complete **multimodal workflow**.
- Include a **workflow/architecture diagram or comparison table**.
- Mention a **real-world application** of the implemented solution.
- Explain the **strengths and limitations** of the approach.
- Include proper **references/citations** for the libraries, APIs, models, or research sources used.

Code of Conduct

I Karthik/192472281 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: karthik _____

SECTION A : Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)

Text-to-Image Programming

1. Write a Python program that accepts a text prompt from the user and generates an image using a text-to-image AI model.
2. Write a Python program that generates an image for the prompt “**A futuristic smart classroom with AI robots.**”
3. Modify a text-to-image program so that the user can enter the prompt dynamically through the keyboard.
4. Write a program that accepts **three different prompts** and generates an image for each prompt.
5. Write a Python program that saves the generated image with a filename provided by the user.
6. Modify the program to display the generated image after generation.
7. Write a program that generates an image from the prompt and stores it in a specified output folder.
8. Write a program that handles an empty text prompt and displays an appropriate error message.
9. Modify a text-to-image program to allow the user to repeatedly generate images until they select **Exit**.
10. Write a program that takes a short description such as “**a cat playing with a robot**” and generates an image from it.

Speech-to-Text Programming

11. Write a Python program that captures audio from the microphone and converts the user's speech into text.
12. Write a program that records the user's voice for a specified duration and performs speech-to-text conversion.
13. Modify an STT program to display the recognized text on the screen.
14. Write a program that converts the spoken sentence “**Artificial intelligence is changing education**” into text.
15. Write a program that continuously listens to the microphone and displays the recognized speech.
16. Modify the STT program to handle the situation where no speech is detected.
17. Write a program that handles speech-recognition errors without terminating the application.
18. Write a program that saves the recognized speech into a .txt file.
19. Modify an STT program to recognize speech in a language selected by the user.
20. Write a program that accepts speech input and counts the number of words recognized.

Text-to-Speech Programming

21. Write a Python program that converts a user-entered sentence into speech.
22. Write a program that converts “**Welcome to the Multimodal AI Laboratory**” into speech.
23. Modify a TTS program so that the user can enter any text dynamically.
24. Write a program that reads the contents of a .txt file aloud using TTS.
25. Modify the TTS program to allow the user to change the speech rate.
26. Write a program that provides a menu for selecting different available voices.
27. Write a program that converts multiple sentences into speech sequentially.
28. Modify the program so that the generated speech is saved as an audio file.
29. Write a program that checks whether the input text is empty before performing TTS.
30. Write a program that reads the following text aloud:
“**Generative AI can create text, images, audio, and other forms of content.**”

. Integrated Multimodal Programming

31. Write a Python program that accepts **text** → **generates an image** → **displays the generated image**.
32. Write a program that accepts **speech** → **converts speech to text** → **displays the recognized text**.
33. Write a program that accepts **text** → **converts it to speech** → **plays the generated audio**.
34. Write a program that accepts **speech** → **converts it to text** → **converts the resulting text back into speech**.
35. Write a program implementing the pipeline:
Speech → **Text** → **Image**.
36. Write a program implementing the pipeline:
Text → **Image** + **Speech**.
37. Write a menu-driven Python application with three options:
**1. Text-to-Image
38. Speech-to-Text
39. Text-to-Speech**
40. Write a complete mini-demo in which the user **speaks a description**, the program converts it into text, and the recognized text is used as the **prompt for image generation**.

29. Write a program that checks whether the input text is empty before performing TTS.

1. Requirement

Develop a Text-to-Speech (TTS) program that checks whether the user has entered any text before converting it into speech. If the input is empty, the program should display an appropriate warning message.

2. Aim

To implement a Text-to-Speech application that validates the user input and prevents speech generation when the text input is empty.

3. Working

- The user enters text into the application.
- The user clicks the **Convert Text to Speech** button.
- The program checks whether the text input is empty.
- If the input is empty, a warning message is displayed.
- If valid text is entered, the TTS engine converts the text into speech.
- The generated audio is played in the application.

4. Folder and File Structure

UNIT_IV_Multimodal_Generative_AI

└── Q29_Check_Empty_Text

 └── text_to_speech.py

5. Implementation Code

Python

Run

```
import streamlit as st
```

```
from gtts import gTTS
```

```
import tempfile
```

```
st.title("🔊 Text-to-Speech Application")
```

```
text = st.text_area(
```

```
    "Enter text to convert into speech:"
```

```
)
```

```
if st.button("Convert Text to Speech"):
```

```

# Check whether the input is empty
if not text.strip():
    st.warning(" ⚠ Please enter some text before performing Text-to-Speech.")

else:
    tts = gTTS(text=text, lang="en")

    with tempfile.NamedTemporaryFile(
        delete=False, suffix=".mp3"
    ) as audio_file:

        tts.save(audio_file.name)

    st.success(" ✅ Speech generated successfully!")

    st.audio(
        audio_file.name,
        format="audio/mp3"
    )

```

6. Input

Test Case 1: Empty Input

(No text entered)

Test Case 2: Valid Input

Artificial Intelligence is changing the world.

7. Output

For Empty Input:

Please enter some text before performing Text-to-Speech.

For Valid Input:

Speech generated successfully!

The application generates and plays the corresponding audio.

8. Algorithm

START

Display Text-to-Speech application

Accept text input from the user

```
IF text is empty THEN
    Display warning message
ELSE
    Convert text into speech
    Generate audio file
    Play generated audio
END IF
```

STOP

9. Actual Implementation Screenshots

Figure 1: Text-to-Speech application with an empty input field displaying a warning message.

Figure 2: Text-to-Speech application with valid text and successfully generated audio.

10. Result

Question 29 was successfully implemented. The application checks whether the user has entered text before performing Text-to-Speech conversion. When the input is empty, an appropriate warning message is displayed. When valid text is provided, the application successfully generates speech.

11. Conclusion

The program successfully validates user input before performing Text-to-Speech conversion. This prevents unnecessary processing of empty input and improves the reliability and user experience of the application.

39. Text-to-Speech

1. Requirement

Develop the Text-to-Speech option of a menu-driven multimodal Python application. The program should accept text from the user and convert it into speech. This corresponds to the Text-to-Speech option listed under the integrated multimodal programming questions.

2. Aim

To develop a Python application that converts user-entered text into audible speech using Text-to-Speech technology.

3. Working

- The user selects the Text-to-Speech option.
- The application provides a text input area.
- The user enters the required text.
- The program checks whether the input is empty.
- The entered text is converted into speech.
- The generated audio is played in the application.

4. Folder and File Structure

```
UNIT_IV_Multimodal_Generative_AI
├── Q39_Text_To_Speech
│   └── text_to_speech.py
```

5. Implementation Code

Python

Run

```
import streamlit as st
```

```
from gtts import gTTS
```

```
import tempfile
```

```
st.title("🔊 Text-to-Speech Application")
```

```
text = st.text_area(
    "Enter text to convert into speech:",
    placeholder="Type your text here..."
)
```

```
if st.button("🔊 Convert Text to Speech"):
```

```

if not text.strip():
    st.warning(" ⚠ Please enter some text.")

else:
    tts = gTTS(text=text, lang="en")

    with tempfile.NamedTemporaryFile(
        delete=False,
        suffix=".mp3"
    ) as audio_file:

        tts.save(audio_file.name)

    st.success(" ✅ Speech generated successfully!")

    st.audio(
        audio_file.name,
        format="audio/mp3"
    )

```

6. Input

Example text:

Welcome to the Multimodal AI Laboratory.

7. Output

The application converts the entered text into speech and provides an audio player.

Speech generated successfully!

8. Algorithm

START

Display Text-to-Speech application

Accept text input from the user

IF the text is empty

 Display warning message

ELSE

 Convert text into speech

Save generated speech as an audio file

Play the audio

END IF

STOP

9. Actual Implementation Screenshots

Figure 1: Text-to-Speech application displaying the text input area.

Figure 2: User entering text into the application.

Figure 3: Generated speech audio successfully displayed and played.

10. Real-World Application

Text-to-Speech technology can be used in:

- Accessibility applications for visually impaired users.
- Audiobooks and digital learning platforms.
- Voice assistants.
- Educational applications.
- Automated announcement systems.

11. Strengths and Limitations

Strengths:

- Simple and easy to use.
- Converts text into audible speech quickly.
- Useful for accessibility and education.
- Supports automated voice generation.

Limitations:

- Requires an internet connection when using some TTS services.
- Voice customization may be limited.
- Pronunciation may not always be accurate for technical terms.

12. Result

Question 39 was successfully implemented. The Text-to-Speech application accepts text entered by the user and successfully converts it into audible speech.

13. Conclusion

The Text-to-Speech module was successfully implemented as part of the multimodal application.

It provides an interactive method for converting written text into speech and demonstrates the practical use of Text-to-Speech technology in engineering and AI applications.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Pipeline Implementation	30%	Correctly implements a working text-to-image AND speech-to-text/text-to-speech pipeline	Both implemented with minor issues	Only one modality implemented correctly	Neither modality functions correctly
Output Quality & Relevance	25%	Generated outputs are high-quality and clearly relevant to inputs	Outputs are mostly relevant	Outputs are of inconsistent quality	Outputs are largely irrelevant or poor quality
Input Robustness	20%	Handles varied inputs robustly (different prompts/audio clips)	Handles common inputs well	Handles only the simplest inputs	Fails on basic inputs
Demo Polish	15%	Polished, engaging demo of both modalities	Clear demo with minor polish issues	Basic or rough demo	No usable demo presented
Tool/Model Explanation	10%	Clear explanation of models/tools used and their limitations	Adequate explanation	Minimal explanation	No explanation provided

Marks Evaluation:

Question No.	Pipeline Implementation (30%)	Output Quality & Relevance (25%)	Input Robustness (20%)	Demo Polish (15%)	Tool/Model Explanation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						

